Practicum No.15 **Date:…….**

Bitwise Logical Operations (CPU vs GPU)

I. Practical Significance

The experiment using the NVIDIA Jetson Nano demonstrates key concepts of parallel computing by comparing CPU's sequential execution with GPU's **parallel processing**. Through **bitwise logical operations (AND, OR, XOR)**, students understand how low-level binary operations can be executed efficiently on large datasets. It highlights how GPU cores perform operations simultaneously, improving computational efficiency. The experiment also introduces memory transfer between CPU and GPU using CuPy and enables performance analysis. Overall, it strengthens understanding **of GPU computing, binary operations, and real-time embedded applications.**

II. Competency and Industry-Specific Practical Skills

This practicum exercise is expected to develop the following skills for the industry-identified competency.

- ***Apply*** GPU computing principles to optimize bitwise logical operations using embedded systems
  a) **Identify** bitwise operations suitable for parallel execution using the NVIDIA Jetson Nano (PDO)
  b) **Implement** CPU and GPU-based bitwise operations using NumPy and CuPy (CDO)
  c) **Analyze** execution efficiency and validate correctness of GPU acceleration (PDO)

III. Relevant CO(s) *(As stated in the Course Structure)*

- **CO1. *Evaluate*** GPU system architectures to enhance high-performance computing solutions.

IV. Practical Outcome (PrO) *(As stated in the Course Structure)*

- ***Develop*** instruction-level parallelism (ILP) techniques to efficiently perform **bitwise logical operations (AND, OR, XOR)** for the given set of instructions

V. Related ADO(s) *(As stated in the Course Structure)*

a) ***Follow*** safe computing and coding practices
b) ***Practice*** teamwork and documentation discipline
c) ***Comply*** with ethical coding conventions

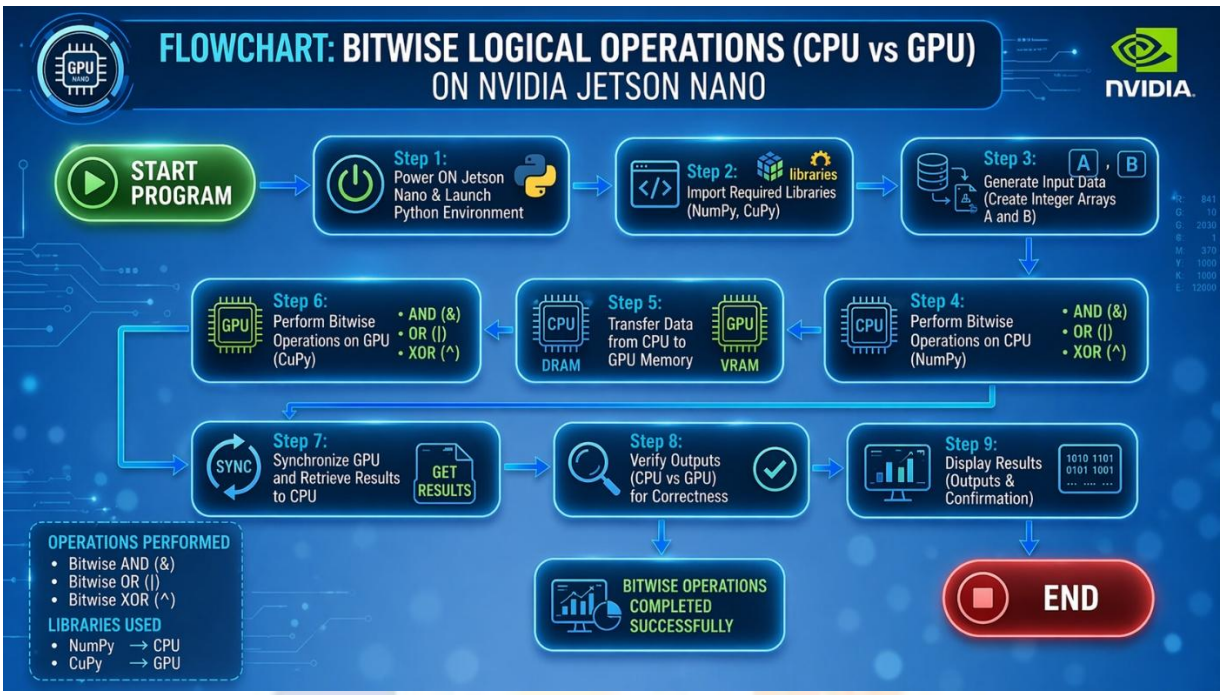
VI. Minimum Underpinning Theory *(Include relevant content & images for this practicum.)*

Bitwise logical operations are fundamental operations performed directly on the binary representation of data. Operations such as AND, OR, and XOR are widely used in digital systems, encryption, and low-level programming. In CPU execution, these operations are performed sequentially, whereas GPU execution enables parallel processing of multiple data elements simultaneously. The NVIDIA Jetson Nano provides an efficient platform for accelerating such operations using its integrated GPU. Libraries like CuPy allow seamless GPU-based computation, while NumPy is used for CPU-based operations, enabling performance comparison.

VII. Practicum Set-up/Circuit Diagram/Algorithm/Flowchart *(Include probable photos of experimental set-up.)*

Students must have:

- NVIDIA Jetson Nano with power supply and SD card
- JetPack SDK installed (Ubuntu OS with CUDA support)
- Python installed
- Required libraries: NumPy (CPU) and CuPy (GPU)
- Script file: gpu_bitwise.py
- Input dataset (integer arrays)
- Basic knowledge of Python and binary operations



VIII. Resources Required

S. No.	Resource	Suggested Broad Specification	Quantity
1	NVIDIA Jetson Nano	Quad-core ARM CPU, 4GB RAM, integrated GPU, microSD storage	1 No.
2	Power Supply & Accessories	5V/4A power adapter, HDMI monitor, keyboard, mouse	1 Set.
3	JetPack SDK	Ubuntu OS with CUDA, cuDNN support	1 No.
4	Python	Python 3.x environment	1 No.
5	NumPy	CPU-based numerical computation library	1 No.
6	CuPy	GPU-accelerated array computation library	1 No.
7	Internet Connection	For installation of required libraries and updates	1 No.

IX. Safety Precautions

a) ***Save*** work frequently and maintain backup copies of your Python scripts.
b) ***Comment*** the code properly and include a header with author name, date, input size, and execution details.
c) ***Close*** unnecessary applications on the NVIDIA Jetson Nano to ensure sufficient memory and better GPU performance.
d) ***Validate*** input data (vector size, data type) to avoid runtime errors and memory overflow during CPU and GPU execution.

X. Procedure *(Self-Instructional)*

a) ***Execute*** both CPU and GPU operations for comparison
b) ***Generate*** input arrays automatically
c) ***Perform*** bitwise operations using NumPy and CuPy
d) ***Save*** the program and verify the code
e) ***Validate*** the inputs for correct numeric format and ensure compatibility with the selected execution mode on the NVIDIA Jetson Nano.

f) ***Perform*** bitwise logical operations (AND, OR, XOR) using the selected mode (CPU with NumPy or GPU with CuPy)
g) ***Display*** the computed output results clearly in the console.
h) ***Allow*** the user to repeat the execution with different input sizes or modes for comparison.
i) ***Terminate*** the program when the user chooses to exit.

Step 1: Boot Jetson Nano

- Insert SD card with JetPack SDK
- Power ON the device
- Complete initial setup

Step 2: Install Required Libraries

Open Terminal and type:
sudo apt update
pip3 install numpy
pip3 install cupy-cuda11x

Step 3: Create Python Program

Create a file:
nano gpu_bitwise.py

```
# Ex No 15: Bitwise Logical Operations (CPU vs GPU)

import numpy as np
import cupy as cp

size = 10**6 # input size

# Generate integer arrays
a = np.random.randint(0, 10, size)
b = np.random.randint(0, 10, size)

# ================= CPU Execution =================
and_cpu = np.bitwise_and(a, b)
or_cpu = np.bitwise_or(a, b)
xor_cpu = np.bitwise_xor(a, b)

# ================= GPU Execution =================
a_gpu = cp.array(a)
b_gpu = cp.array(b)

and_gpu = cp.bitwise_and(a_gpu, b_gpu)
or_gpu = cp.bitwise_or(a_gpu, b_gpu)
xor_gpu = cp.bitwise_xor(a_gpu, b_gpu)

print("Bitwise operations completed")
```

Step 4: Save the program

Step 5: Run the Program
python3 gpu_bitwise.py

Sample Output
Sample Input:
A = [5, 3, 7, 2, 6]
B = [1, 2, 3, 4, 5]

Expected Output:
Bitwise AND (CPU): [1, 2, 3, 0, 4]
Bitwise OR (CPU): [5, 3, 7, 6, 7]
Bitwise XOR (CPU): [4, 1, 4, 6, 3]
Bitwise AND (GPU): [1, 2, 3, 0, 4]
Bitwise OR (GPU): [5, 3, 7, 6, 7]
Bitwise XOR (GPU): [4, 1, 4, 6, 3]

The experiment successfully demonstrated bitwise logical operations using CPU and GPU, confirming correct results and efficient parallel execution.

XI. Actual Resources Used (*To be filled by the students*)

S. No.	Name of Resource	Broad Specifications		Quantity	Remarks (If any)
		Make	Details		
1.	NVIDIA Jetson Nano	NVIDIA	Quad-core ARM CPU, 4GB RAM, integrated GPU	1	Used for GPU execution
2.	Power Supply & Accessories	Generic	Generic 5V/4A adapter, HDMI monitor, keyboard, mouse	1 Set	Required for setup
3.	JetPack SDK	NVIDIA	Ubuntu OS with CUDA support	1	Pre-installed
4.	Python with NumPy & CuPy	Open Source	Python 3.x with required libraries	1	Used for implementation

XII. Actual Procedure Followed (*To be filled by the students*)

1. **Booted** the NVIDIA Jetson Nano and initialized the **Python execution environment**.
2. **Installed** the required libraries such as **NumPy (CPU computation)** and **CuPy (GPU computation)** using terminal commands.
3. **Created** a Python script (gpu_bitwise.py) specifically for performing **bitwise logical operations**.
4. **Generated** random integer input arrays **A and B**, ensuring values are suitable for **bitwise operations**.
5. **Executed** bitwise operations (**AND, OR, XOR**) using **CPU (NumPy)** in a sequential manner.
6. **Transferred** the input data from **CPU memory to GPU memory** using CuPy arrays.
7. **Performed** the same bitwise operations using **GPU (CuPy)** leveraging **parallel processing**.
8. **Retrieved** results from GPU memory back to CPU (if required) for verification.
9. **Compared** CPU and GPU outputs to ensure **accuracy and correctness**.
10. **Observed** execution behavior and noted performance characteristics.
11. **Repeated** the experiment for consistency and validation of results.

XIII. Observations and Dimensional Measurement *(To be filled by the students)*

The experiment was carried out on the NVIDIA Jetson Nano by executing **bitwise logical operations (AND, OR, XOR)** using both CPU with NumPy and GPU with CuPy. It was observed that the operations were successfully executed for large input datasets without any runtime errors. The **outputs obtained from CPU and GPU were identical**, confirming the correctness of implementation.

For smaller input sizes, the execution difference between CPU and GPU was minimal, as bitwise operations are relatively simple and fast even on CPU. However, for larger datasets, the GPU demonstrated **better efficiency due to parallel execution across multiple cores**. The experiment also highlighted that bitwise operations are **low-level and computationally efficient**, making them suitable for parallel execution. Minor variations in execution performance were observed due to system load and memory usage.

XIV. Results/Observations *(To be filled by the students)*

1. The **bitwise AND, OR, and XOR operations** were successfully executed on both CPU and GPU.
2. The **outputs from CPU and GPU were identical**, confirming correctness.
3. GPU execution demonstrated **efficient handling of large datasets**.
4. CPU performed operations sequentially, while GPU executed them in **parallel threads**.
5. Bitwise operations showed **low computational complexity**, resulting in fast execution.
6. GPU utilization improved overall performance for large-scale inputs.
7. Data transfer between CPU and GPU had **minimal impact** on performance in this case.
8. The experiment validated the suitability of GPU for **parallel binary operations**.
9. Execution consistency was observed across multiple runs.
10. The GPU proved effective for **high-throughput logical computations**.

XV. Interpretation of Results *(Students to state the meaning of the above-obtained results/observations)*

The results indicate that **GPU-based execution using CuPy is highly effective** for performing bitwise logical operations on large datasets. While CPU executes operations sequentially, GPU processes multiple data elements simultaneously using **parallel architecture**, leading to improved efficiency. The identical outputs from CPU and GPU confirm that GPU computation is **accurate and reliable**.

Since bitwise operations are inherently simple and operate at the binary level, the performance gain is moderate compared to arithmetic-heavy computations. However, the ability of GPU to process large volumes of data concurrently still provides a noticeable advantage in large-scale applications. The experiment also demonstrates that **parallel processing is beneficial even for simple operations when applied to large datasets**, making GPU acceleration valuable in embedded and real-time systems.

XVI. Conclusions *(Students to draw conclusions (or take decisions) based on the interpretation of results*

The experiment successfully demonstrated the implementation of **bitwise logical operations (AND, OR, XOR)** using both CPU and GPU on the NVIDIA Jetson Nano platform. The results confirmed that GPU-based execution using CuPy produces outputs that are **accurate and identical** to CPU-based execution using NumPy.

It was observed that GPU execution improves performance by utilizing **parallel processing capabilities**, especially when handling large datasets. While CPU executes operations sequentially, GPU divides the workload across multiple cores, reducing overall computation time. Although bitwise operations are simple, the experiment highlights how GPU acceleration can still enhance performance in large-scale data processing.

In conclusion, GPU-based computation is highly effective for performing **high-speed logical operations**, making it suitable for applications such as **digital systems, encryption, data processing, and embedded computing**. The experiment reinforces the importance of parallel computing in modern computing systems and demonstrates the efficiency of GPU architectures.

The experiment validates that GPU-based parallel execution enhances the efficiency of bitwise logical operations while maintaining accuracy and reliability.

XVII. Suggested Practicum Related Questions:

Note: *Below are a few questions related to industry-specific higher-order thinking skills (HOT) that teachers must use to ensure students achieve the predetermined course outcomes.*

1. Implement bitwise AND, OR, and XOR operations using CPU (NumPy) and GPU (CuPy) and compare their performance.

Implement bitwise AND, OR, XOR (CPU vs GPU) & compare performance
CODE:

```
import numpy as np
import cupy as cp
import time

size = 10**6

# Generate random integer data
a = np.random.randint(0, 10, size)
b = np.random.randint(0, 10, size)

# ================== CPU Execution ==================
start = time.time()
and_cpu = np.bitwise_and(a, b)
or_cpu  = np.bitwise_or(a, b)
xor_cpu = np.bitwise_xor(a, b)
cpu_time = time.time() - start

# ================== GPU Execution ==================
a_gpu = cp.array(a)
b_gpu = cp.array(b)

start = time.time()
and_gpu = cp.bitwise_and(a_gpu, b_gpu)
or_gpu  = cp.bitwise_or(a_gpu, b_gpu)
xor_gpu = cp.bitwise_xor(a_gpu, b_gpu)
cp.cuda.Stream.null.synchronize()
gpu_time = time.time() - start

print("CPU Time:", cpu_time)
print("GPU Time:", gpu_time)
```

Theory
This program performs **bitwise logical operations** using both **CPU (NumPy)** and **GPU (CuPy)**. The CPU executes operations sequentially, while the GPU executes them in **parallel across multiple cores**. For large datasets, GPU execution becomes faster due to parallelism, demonstrating improved **computational efficiency**.

2. Analyze how input size affects execution time in bitwise operations on CPU and GPU.
Analyze performance with different input sizes

CODE:

```
import numpy as np
import cupy as cp
import time

sizes = [10**3, 10**5, 10**6]

for size in sizes:
   a = np.random.randint(0, 10, size)
   b = np.random.randint(0, 10, size)

   # CPU
   start = time.time()
   np.bitwise_and(a, b)
   np.bitwise_or(a, b)
   np.bitwise_xor(a, b)
   cpu_time = time.time() - start

   # GPU
   a_gpu = cp.array(a)
   b_gpu = cp.array(b)

   start = time.time()
   cp.bitwise_and(a_gpu, b_gpu)
   cp.bitwise_or(a_gpu, b_gpu)
   cp.bitwise_xor(a_gpu, b_gpu)
   cp.cuda.Stream.null.synchronize()
   gpu_time = time.time() - start

print("Size:", size, "CPU:", cpu_time, "GPU:", gpu_time)
```

Theory

This experiment studies how **execution time changes with input size**. For small inputs, CPU and GPU times are similar because of **data transfer overhead**. As input size increases, GPU performance improves significantly due to **parallel processing**, showing better **scalability** compared to CPU.

3. Demonstrate data transfer between CPU and GPU memory and evaluate its impact on bitwise computation performance.

Demonstrate CPU–GPU data transfer impact

CODE:

```
import numpy as np
import cupy as cp
import time

size = 10**6

a = np.random.randint(0, 10, size)

# Measure transfer time (CPU → GPU)
start = time.time()
a_gpu = cp.array(a)
transfer_time = time.time() - start

# Perform GPU operation
start = time.time()
result = cp.bitwise_and(a_gpu, a_gpu)
cp.cuda.Stream.null.synchronize()
```

```
    gpu_exec_time = time.time() - start

    print("Transfer Time:", transfer_time)
print("GPU Execution Time:", gpu_exec_time)
```

Theory:
In GPU computing, data must be transferred from **CPU memory (RAM)** to **GPU memory (VRAM)** before execution. This introduces **transfer overhead**, which affects performance for small datasets. However, for large datasets, the speed of **parallel execution outweighs transfer cost**, making GPU computation more efficient overall.

GPU provides better performance for large-scale bitwise operations due to parallel processing, but data transfer overhead affects performance for smaller inputs.

XVIII References / Suggestions for further reading

a) NVIDIA Developer – Jetson Nano Documentation: https://developer.nvidia.com/embedded/jetson-nano-developer-kit
b) CUDA Toolkit Documentation: https://docs.nvidia.com/cuda/
CuPy Official Documentation: https://docs.cupy.dev/en/stable/
NumPy Official Documentation: https://numpy.org/doc/
c) GPU Computing Basics – YouTube – NVIDIA Developer
d) Parallel Computing with Python (NumPy vs CuPy) – YouTube Tutorials

XIX Suggested Assessment Scheme

The performance indicators given serve as a guideline for assessing the *'process-related skills/LOs'* (marks to be awarded in real-time in the laboratory by the faculty member, as these cannot be measured after the practicum is over) and *'product-related skills/LOs'*

Note: The weightage for the ***process-related skills*** and ***product-related skills*** will change depending on the PrO, COs, and the competency related to the concerned course.

Performance Indicators		Maximum Marks	Marks Obtained
Process-related Skills: 18 Marks - 60%			
1	Correct usage of Python and handling input data	6	
2	Correct implementation of CPU (NumPy) and GPU (CuPy) execution	5	
3	Safe practices, proper coding and documentation	4	
4	Team participation	3	
Product-related Skills: 12 Marks - 40%			
1	Accurate EA results	3	
2	Correct interpretation	2	
3	Quality of outputs (tables/plots)	3	
4	Conclusion quality	2	
5	HOT question responses	1	
6	Journal submission	1	
	Total	**30**	

To be filled by Student:

S.No.	Name of the Student	Register No.
1	**Neavis Rishva. J**	**URK25CS1236**

To be filled by the Faculty Member:

Marks Obtained			Name and Designation of the Faculty Member	Date of Evaluation
Process-Related Skills (18)	**Product-Related Skills (12)**	**Total (30)**		

******

ARISE
AND
SHINE
Karunya